

INGENIERÍA DE SISTEMAS
LENGUAJE DE PROGRAMACIÓN - 2510B04G2

Seguimiento 1 (5%)

JEFFREY ALEXANDER AGUDELO ESPITIA

CC: 1094955981

TULUÁ - VALLE DEL CAUCA



2025

Foro de Discusión: Dependencias en Programación

Pregunta de Discusión

¿Qué es una dependencia en el contexto de programación? ¿Por qué son importantes para las aplicaciones? Explique y justifique sus respuestas.

Aporte Principal

Dependencias: Los Cimientos de la Construcción de Software Moderno

En el desarrollo de software contemporáneo, las **dependencias** representan componentes, bibliotecas o módulos externos que una aplicación requiere para funcionar correctamente. Estas pueden ser paquetes de código reutilizable, frameworks, librerías de terceros o incluso otros servicios que proporcionan funcionalidades específicas sin necesidad de implementarlas desde cero (Fowler, 2019).

¿Qué es una Dependencia?

Una dependencia es cualquier recurso externo del que depende nuestro código para ejecutarse. Por ejemplo, si desarrollamos una aplicación web con React, esta biblioteca es una dependencia de nuestro proyecto. Del mismo modo, si utilizamos Axios para realizar peticiones HTTP, Axios se convierte en otra dependencia (Mozilla Developer Network, 2024).

Las dependencias se clasifican principalmente en dos tipos:

- **Dependencias de producción:** Necesarias para que la aplicación funcione en el entorno final (ej: Express, React, Lodash)
- **Dependencias de desarrollo:** Utilizadas solo durante el proceso de desarrollo (ej: Jest para pruebas, ESLint para análisis de código, Webpack para empaquetado)

Importancia de las Dependencias

Las dependencias son fundamentales en el desarrollo moderno por varias razones críticas:

1. Reutilización de Código y Eficiencia

En lugar de reinventar la rueda, los desarrolladores pueden aprovechar soluciones probadas y optimizadas. Por ejemplo, implementar un sistema de autenticación seguro desde cero

requeriría semanas de desarrollo y pruebas exhaustivas. Utilizar una biblioteca como Passport.js permite integrar autenticación robusta en horas, con la confianza de que ha sido probada por miles de desarrolladores (npm, Inc., 2024).

2. Mantenimiento y Actualizaciones

Las dependencias bien mantenidas reciben actualizaciones constantes que incluyen correcciones de seguridad, mejoras de rendimiento y nuevas características. Esto significa que nuestra aplicación se beneficia automáticamente de estos avances sin necesidad de implementarlos internamente (GitHub, Inc., 2024).

3. Especialización y Calidad

Las bibliotecas populares son desarrolladas y mantenidas por expertos en áreas específicas. Por ejemplo, Moment.js (ahora Day.js) para manejo de fechas, Chart.js para visualización de datos, o Socket.io para comunicación en tiempo real. Estas herramientas han sido refinadas a través de años de uso en producción y feedback de la comunidad (Stack Overflow, 2024).

4. Aceleración del Desarrollo

El tiempo de desarrollo se reduce significativamente al no tener que implementar funcionalidades comunes. Un equipo puede enfocarse en la lógica de negocio específica de su aplicación mientras delega tareas genéricas a dependencias confiables (Martin, 2018).

Gestión de Dependencias: Un Aspecto Crítico

Sin embargo, el uso de dependencias también introduce desafíos que deben gestionarse adecuadamente:

- **Versionado:** Es crucial especificar versiones compatibles para evitar conflictos. Herramientas como npm, yarn o pip utilizan archivos de configuración (package.json, requirements.txt) para mantener un registro preciso de las dependencias y sus versiones (Node.js Foundation, 2024).
- **Seguridad:** Las dependencias pueden contener vulnerabilidades. Es fundamental realizar auditorías periódicas usando herramientas como npm audit, Snyk o Dependabot para identificar y corregir problemas de seguridad (Snyk Ltd., 2024).
- **Tamaño del proyecto:** Cada dependencia añade peso a la aplicación. Es importante evaluar si realmente necesitamos una biblioteca completa o si podemos implementar una solución más ligera (Webpack Contributors, 2024).
- **Mantenimiento:** Dependencias abandonadas o sin mantenimiento pueden convertirse en un riesgo. Es recomendable verificar la actividad del proyecto antes de adoptarlo (Open Source

Initiative, 2024).

Ejemplo Práctico

Consideremos un proyecto Node.js típico. El archivo `package.json` podría incluir:

```
{
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.3",
    "jsonwebtoken": "^9.0.0"
  },
  "devDependencies": {
    "jest": "^29.5.0",
    "nodemon": "^2.0.22"
  }
}
```

Aquí, Express proporciona el framework web, Mongoose facilita la interacción con MongoDB, y JWT maneja la autenticación. En desarrollo, Jest permite escribir pruebas y Nodemon reinicia automáticamente el servidor al detectar cambios.

Reflexión Final

Las dependencias son un pilar fundamental del desarrollo de software moderno. Permiten construir aplicaciones complejas de manera eficiente, aprovechando el conocimiento colectivo de la comunidad de desarrolladores. Sin embargo, su uso debe ser consciente y estratégico, evaluando siempre el balance entre los beneficios que aportan y los riesgos que introducen.

La clave está en mantener un equilibrio: utilizar dependencias cuando aporten valor real, pero sin caer en la "dependencia de dependencias" que puede hacer nuestro proyecto frágil y difícil de mantener. Como desarrolladores, debemos ser curadores cuidadosos de las herramientas que incorporamos en nuestros proyectos.

Preguntas para Profundizar la Discusión

- ¿Cuál es su criterio para decidir si implementar una funcionalidad desde cero o usar una dependencia externa?
- ¿Han experimentado problemas con dependencias obsoletas o vulnerables? ¿Cómo los resolvieron?
- ¿Qué herramientas utilizan para gestionar y auditar las dependencias en sus proyectos?

- ¿Consideran que existe un número "ideal" de dependencias para un proyecto, o depende completamente del contexto?

Comentario al Aporte de un Compañero

Respuesta al aporte de Julián Zapata Rojas

Julián, tu aporte proporciona una clasificación muy completa de las dependencias en programación. Comparto plenamente tu perspectiva sobre la importancia de las dependencias como facilitadores del desarrollo, y me parece especialmente valiosa tu distinción entre **dependencias directas y transitivas**, ya que este concepto es fundamental pero a menudo subestimado por desarrolladores principiantes.

Tu mención de las **dependencias no utilizadas** conecta directamente con lo que planteé sobre la gestión del tamaño del proyecto. Este es un problema real que he enfrentado en proyectos de producción: dependencias que fueron instaladas para una funcionalidad específica pero que quedaron olvidadas cuando esa funcionalidad cambió o se eliminó. Herramientas como `npm prune` o `depcheck` son esenciales para identificar y eliminar estas dependencias huérfanas que inflan innecesariamente el bundle final.

Sin embargo, me gustaría profundizar en un aspecto que mencionas: los **riesgos de las dependencias**. Planteas cinco categorías de riesgos (compatibilidad, licencia, rendimiento, mantenibilidad y seguridad), pero ¿cuál consideras que es el más crítico en el contexto actual del desarrollo de software? En mi experiencia, las vulnerabilidades de seguridad han tomado protagonismo, especialmente después de incidentes como el de *log4j* en 2021 o el más reciente problema con *node-ipc* en 2022, donde dependencias ampliamente utilizadas introdujeron código malicioso.

Esto me lleva a plantear una pregunta para enriquecer la discusión: **¿Cómo equilibramos la necesidad de mantener dependencias actualizadas (para corregir vulnerabilidades) con el riesgo de introducir breaking changes que puedan romper nuestra aplicación?** Este dilema es particularmente relevante en proyectos legacy donde las actualizaciones pueden ser costosas y arriesgadas.

Además, mencionas los problemas de licencia como un riesgo. Este es un punto crucial que muchos desarrolladores pasan por alto. ¿Has tenido experiencia con conflictos de licencias en tus proyectos? Por ejemplo, usar una dependencia con licencia GPL en un proyecto comercial puede tener implicaciones legales significativas. Herramientas como `license-checker` o `FOSSA` pueden ayudar a auditar las licencias de todas nuestras

dependencias, pero la pregunta es: **¿cuántos equipos realmente realizan esta auditoría de forma sistemática?**

Finalmente, tu clasificación de tipos de dependencias podría expandirse para incluir las **dependencias peer** (peer dependencies), que son especialmente importantes en el ecosistema de plugins y extensiones. Por ejemplo, un plugin de React debe especificar React como peer dependency para evitar tener múltiples versiones de React en el mismo proyecto, lo cual causaría errores graves.

Tu aporte establece bases sólidas para la discusión. ¿Qué estrategias específicas recomiendas para mitigar los riesgos que mencionas, especialmente en proyectos de gran escala?

Referencias Bibliográficas

- Fowler, M. (2019). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
- GitHub, Inc. (2024). *Dependency Management Best Practices*.
<https://docs.github.com/en/code-security/supply-chain-security>
- Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Mozilla Developer Network. (2024). *Understanding Dependencies in JavaScript*.
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing
- Node.js Foundation. (2024). *npm Documentation: Managing Dependencies*.
<https://docs.npmjs.com/cli/v9/configuring-npm/package-json>
- npm, Inc. (2024). *npm Registry Documentation*.
<https://www.npmjs.com/>
- Open Source Initiative. (2024). *The Open Source Definition*.
<https://opensource.org/osd>
- Stack Overflow. (2024). *Developer Survey 2024: Most Popular Technologies*.
<https://survey.stackoverflow.co/2024/>
- Synk Ltd. (2024). *Open Source Security: Best Practices for Dependency Management*.
<https://snyk.io/learn/open-source-security/>
- Webpack Contributors. (2024). *Webpack Documentation: Code Splitting and Bundle Analysis*.
<https://webpack.js.org/guides/code-splitting/>